

Temporal Knowledge Graph GNN Notes

Goals

Model can predict new edges in a graph over time

- This will likely involve merging a GCN (likely attention-based) with a recurrent model, likely an LSTM

Notes

Model will be created with global node pool. Some nodes will be unconnected. This is ok.

First need to consider how to represent the text in edges and nodes

Edge embeddings

- need to be a limited subset that
- embeddings will be learned by the model
- Ex: born, died, friends, enemies, killed_by, etc

Then how to represent node text:

Embedding layer

- Generate separate embeddings for different characteristics using text $\rightarrow$ vector model
 - name
 - description
 - class
 - embeddings will be merged in an embedding layer in the model

The best idea is to go with multiple pretrained embeddings of names, class, and description. Then put an embedding head that takes in each embedding vector, synthesizes it and projects down into GNN dim

- dataset will be a concatenation of the three because we can just leave it as the concat or project down

Text embedding could also be merged with structural embedding in some way, but the structure will probably be learned via convolutions so it is probably ok.

With embedding taken care of each node will be represented by an embedding and we have the starting structure for the graph.

Embeddings layer

- simple MLP that takes all the embedding vectors representing a model and projects them into hidden dim

Update Rule

$$h(t+1)_i = \text{Non-Linearity} \left(\sum (a_{ij} \cdot m_{ij}) \right)$$

where a_{ij} is an attention score and m is message (linear combination of messages)

- attention scores are calculated by

$$a_score_{ij} = \text{Non-Linearity}(a(z_i, z_j, edge_{ij}))$$

where a_{ij} is attention score, a is learned attention function and z is $W_n \cdot h$. Do it only over neighbors

– gotta do softmax to get a_{ij}

- messages (need a way to synthesize edges and nodes)

$$m_{ij} = W_h \cdot h(t)_j + W_r \cdot edge_{ij}$$

- Multi head attentions should be considered after single head is finished. Basically the same thing but project down and concat
 - dim has to be divisible by number of heads
 - Same set of the above equations for each head (so head has separate W_h , W_r , etc scaled down to project into $\text{dim}/\#$ heads)
 - at end concat output of each head for full
 - for efficiency can use the same weights for all heads. Its not as powerful as full multi-head attention, but it still captures different relationships

Further considerations

- since we are using sum should probably use layer norms to keep things from exploding and maybe residuals
- Something like this

$$h(t+1) = \text{GNNLayer}(h(t))$$

$$h(t+1) = \text{LayerNorm}(h(t+1) + h(t))$$

- Note: if using multi head attention need to project $h(t)$ down
- can also add a feedforward network after attention like GNNs

$$h(t+1) = \text{LayerNorm}(h(t+1) + \text{MLP}(h(t+1)))$$

- dropouts could also be useful for better generalization

Running this for a bit will give us rich contextual embeddings for each node, which we can then use for link prediction and node generation

Update LSTM hidden state

- Store a hidden state for each node
 - normal LSTM update conditioned on the node's hidden state
 - Stores much more fine-grained info, at the cost of not having as strong a representation for the graph as a whole
 - use this for scoring

Link prediction

Predict only changes

- All current triples are passed to the next graph
- the model solely predicts additions to the graph
 - make 2 separate scoring heads, one for deletion and one for addition
 - candidates are the true positives (things added or deleted), combined with a equal number of sampled negatives
 - * negs for adding are sampled from candidate proposal strategies
 - node sampling strategies (candidates j for each i)
 - random (probably a bad idea but a bit of noise isnt bad)
 - neighbors (selection of nodes of depth 1–3 away from i)
 - embedding similarity (dot product with node embedding matrix (could be expensive))
 - * edge sampling
 - total random (probably a bad idea but a bit of noise isnt bad)
 - build counts of edges in neighbors and sample probabilistically from it based on count (with noise)
 - count could also be based off how good edge is, so $\text{MLP}(i, j, \text{edge})$ but that might be too expensive
 - TODO: need more possibilities
 - loss is calculated and backpropagated
 - deletions and additions are added

Candidate scoring

- do different scoring heads, one for addition, one for deletion
- batch all triplets into single tensor and pass to scoring MLP
- Feed $(i, j, edge, i_hidden, j_hidden)$ into a basic MLP and get a score
 - condition on LSTM hidden states for more context
 - $MLP(i, j, edge, i_hidden, j_hidden)$
- Apply sigmoid to get probs
- compare against positive add delete samples and calc loss

Loss

- Binary cross entropy with the addition and deletions from next graph
- losses are summed
- positive edges are labeled one and ones that dont exist are 0
- this will hopefully capture loss for both negative and positive samples and for both addition and deletion

Binary cross entropy equation:

$$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Generation

- Autoregressive generation
 - sample candidates based on probabilities output by forward pass
- Start with one graph, run the network over it to get changes, apply changes and pass back in again